



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 26

Project Title: Linux Concurrent Producer–Consumer System Using Semaphores

Team Members

Roll Number	Student Name	Signature
2520090017	Tejasri	
2520090007	Dikshita	
2520090235	Rohitha	

1. Abstract

The **Linux Concurrent Producer–Consumer System Using Semaphores** demonstrates how concurrent processes or threads can safely share resources in Linux. The producer generates data and places it into a shared buffer, while the consumer retrieves and processes it. Without synchronization, concurrent access may cause race conditions, data inconsistency, or buffer overflow and underflow. The project applies **Operating Systems concepts** such as concurrency, synchronization, critical sections, mutual exclusion, shared buffers, POSIX threads, and semaphores. The proposed Linux solution uses a bounded buffer and semaphores to coordinate producers and consumers. The expected outcome is a reliable system demonstrating safe concurrent execution and practical Linux/POSIX system programming concepts.

2. Problem Statement

The project addresses the problem of **safely coordinating concurrent producers and consumers accessing a shared buffer**. Without proper synchronization, simultaneous access can cause race conditions, data inconsistency, buffer overflow, or underflow. This problem is highly relevant to **Operating Systems and Systems Programming** because it demonstrates process/thread concurrency, critical sections, mutual exclusion, resource sharing, and synchronization. The project uses **Linux/POSIX mechanisms and semaphores** to control access to the shared buffer and coordinate producer and consumer execution, providing practical understanding of how operating systems manage concurrent processes and shared resources.

3. Objectives

1. To implement a **concurrent Producer–Consumer system** in Linux.
2. To create a **shared bounded buffer** for communication between producers and consumers.
3. To use **semaphores** for synchronization and controlled access to the shared buffer.

4. To prevent *race conditions, buffer overflow, and buffer underflow*.

Proposed Methodology

The project will be designed and implemented in **Linux using C and POSIX synchronization mechanisms**. A bounded shared buffer will be created between producer and consumer threads. Producers will generate data and place it into the buffer, while consumers will retrieve and process the data. **Semaphores** will control the number of empty and filled buffer slots and ensure synchronized access to the shared resource. The system will use concurrent execution to demonstrate safe resource sharing.

Flow:

Initialize → Create Buffer → Initialize Semaphores → Start Producers & Consumers → Produce Data → Store in Buffer → Consume Data → Synchronize Access → Display Results → Terminate.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
OSIX Threads (pthread_create())	Creates concurrent producer and consumer threads.
Semaphores (sem_init())	Synchronizes access to the shared buffer.
sem_wait()	Waits when the required buffer resource is unavailable.
sem_post()	Signals that a buffer resource has become available.
Mutex (pthread_mutex_lock())	Provides mutual exclusion while accessing the shared buffer.
pthread_mutex_unlock()	Releases the buffer lock after completing an operation

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090017	L.Tejasri	Overall system design, producer implementation, data generation, and project integration
2520090007	S.Dikishita	Consumer implementation, shared buffer management, and semaphore synchronization
2520090235	Rohitha	Testing, debugging, Makefile/Linux setup, documentation, and final demonstration

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system for developing, compiling, and executing the project.
C / C++	Main programming language used to implement the Producer–Consumer system.
GCC Compiler	Compiles the C source code into an executable program

8. Expected Outcome

The project is expected to provide a **Linux-based concurrent Producer–Consumer system** that safely transfers data through a shared bounded buffer. Multiple producer threads will generate data, while consumer threads will retrieve and process the data concurrently.

The system will display the **produced items, consumed items, buffer status, and synchronization activities** during execution. Semaphores and mutexes will ensure that the buffer is accessed safely and prevent race conditions, overflow, and underflow.

The expected outcome is a **working and synchronized concurrent application** that demonstrates practical understanding of Operating Systems concepts such as **concurrency, synchronization, mutual exclusion, critical sections, semaphores, and POSIX threads**.

9. GitHub Repository

GitHub Repository Link: <https://github.com/tejasrilankey/Operating-Systems-2520090017>

Repository Created: _____

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Raghupathi M

Signature:

Remarks:

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____